



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

March 29, 2026

Content in this Practice I

1 Binary Tree

- Representation
- Traversal Methods
- Height
- Number of Leaves

2 Binary Tree (Tasks)

- Number of Leaves
- Maximum Key
- Negative Key

3 Level-Order Traversal

- Levels
- Number of Children
- Negative Key (Level-Order)
- Number of Leaves (Level-Order)

Content in this Practice II

4 Expression Tree

Binary Tree

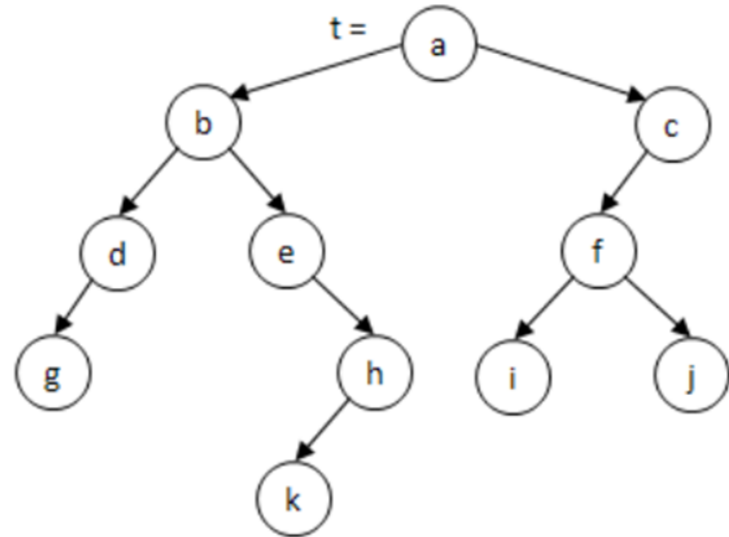
Representation of Binary Trees

- **Linked representation**

- With two pointers: `left`, `right`
- With three pointers: `left`, `right`, `parent`

- **Arithmetic representation**

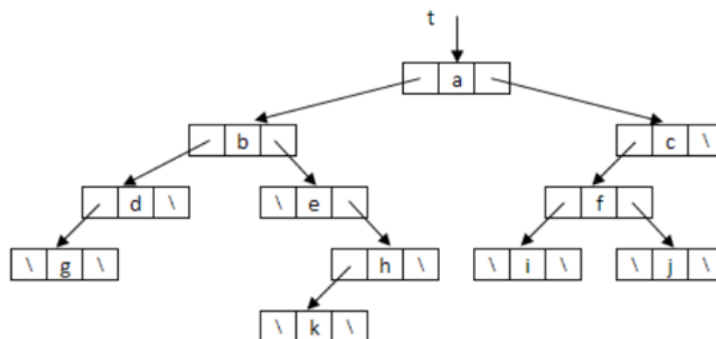
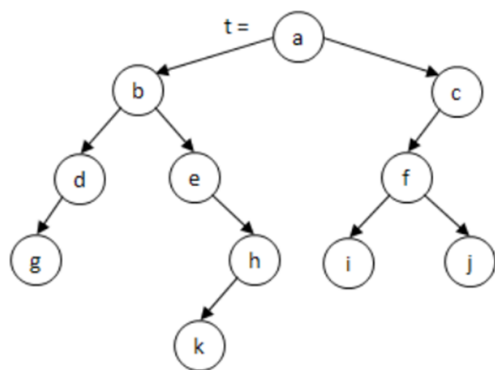
- Level-order arrangement in an array.
- The left child, right child, and parent can be computed from the indices.
- Efficient only for “dense” trees, see heap.
- We will discuss it when covering heaps.



Representation with Two Pointers

Node type:

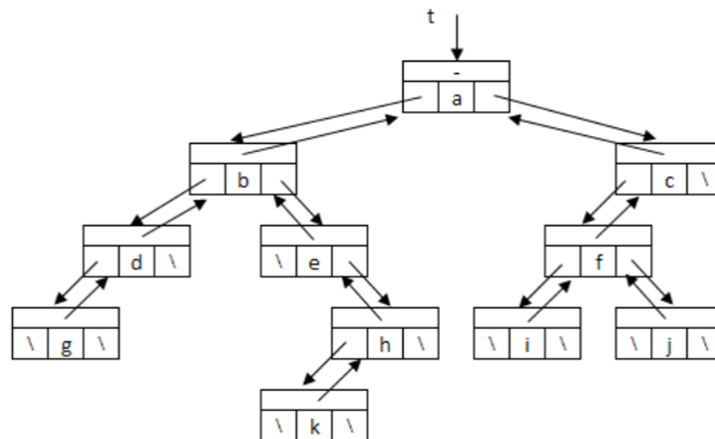
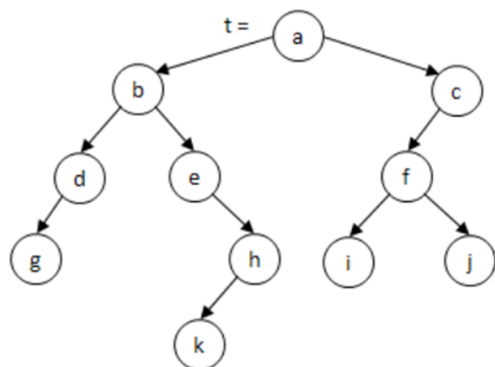
Node
+ $key : \mathcal{T}$ // $\mathcal{T}$ valamilyen ismert típus
+ $left, right : Node^*$
+ $Node() \{ left := right := \emptyset \}$ // egycsúcsú fát képez belőle
+ $Node(x : \mathcal{T}) \{ left := right := \emptyset ; key := x \}$



Representation with Three Pointers

Node type:

Node3
+ $key : \mathcal{T} // \mathcal{T}$ valamilyen ismert típus
+ $left, right, parent : Node3^*$
+ $Node3(p:Node3^*) \{ left := right := \odot ; parent := p \}$
+ $Node3(x : \mathcal{T}, p:Node3^*) \{ left := right := \odot ; parent := p ; key := x \}$

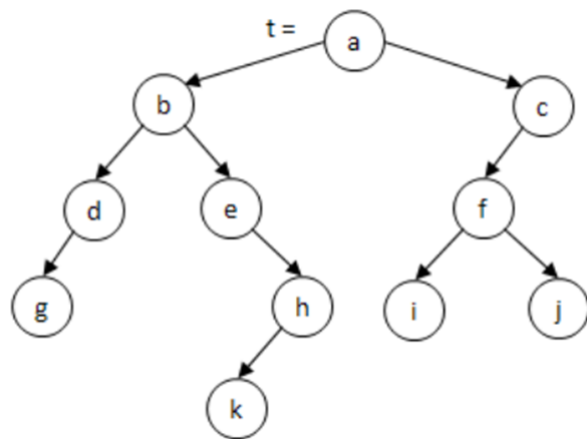


Notations Used in Algorithms

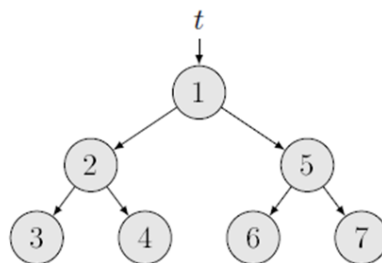
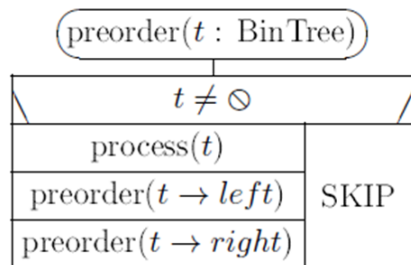
Representation (level operation)	Data structure (level operation)	Meaning
$t \rightarrow left$	$left(t)$	t 's left subtree
$t \rightarrow right$	$right(t)$	t 's right subtree
$t \rightarrow parent$	$parent(t)$	parent of t
$t \rightarrow key$	$key(t)$	key of node t
$t = 0, NIL, NULL$	$t = \Omega$	t is an empty tree

Traversal of Binary Trees

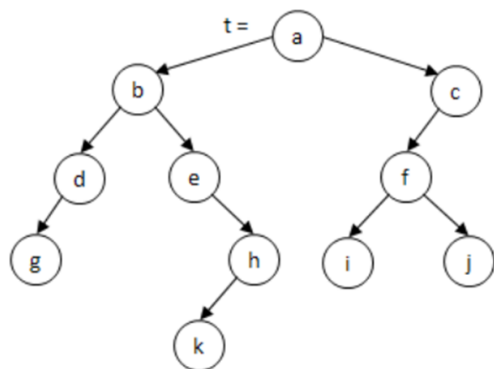
- Traversal means listing the nodes, therefore it can be regarded as enumeration as learned in programming. As we will see later, any enumeration-based programming theorem can be applied using these traversals.
- **Recursive traversal**
 - Preorder
 - Inorder
 - Postorder
- **Level-order traversal:**
 - In case of general graphs, this is equivalent to BFS (breadth-first search).



Preorder Traversal



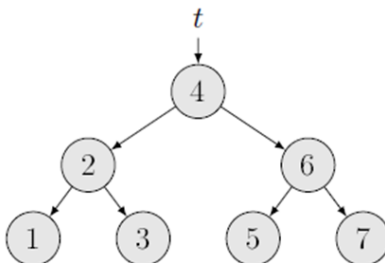
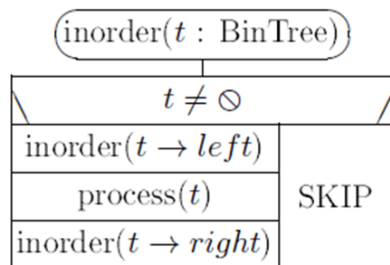
Order in which the elements are accessed.



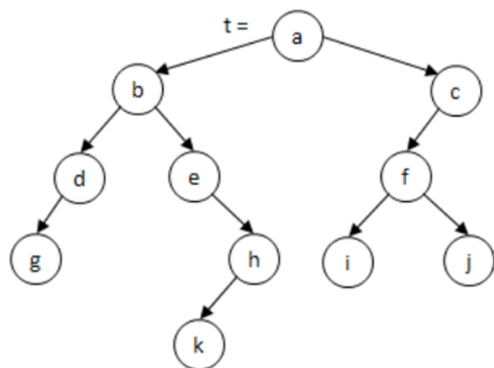
Order obtained by preorder traversal:

a b d g e h k c f i j

Inorder Traversal



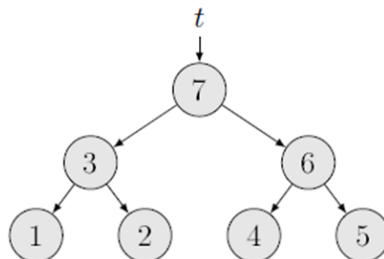
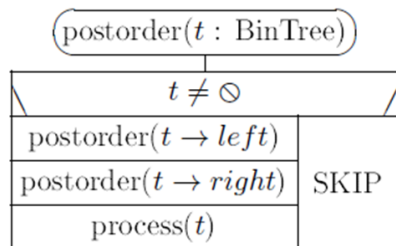
Order in which the elements are accessed.



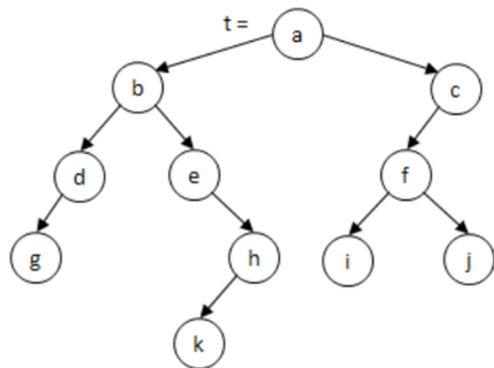
Order obtained by inorder traversal:

g d b e k h a i f j c

Postorder Traversal



Order in which the elements are accessed.

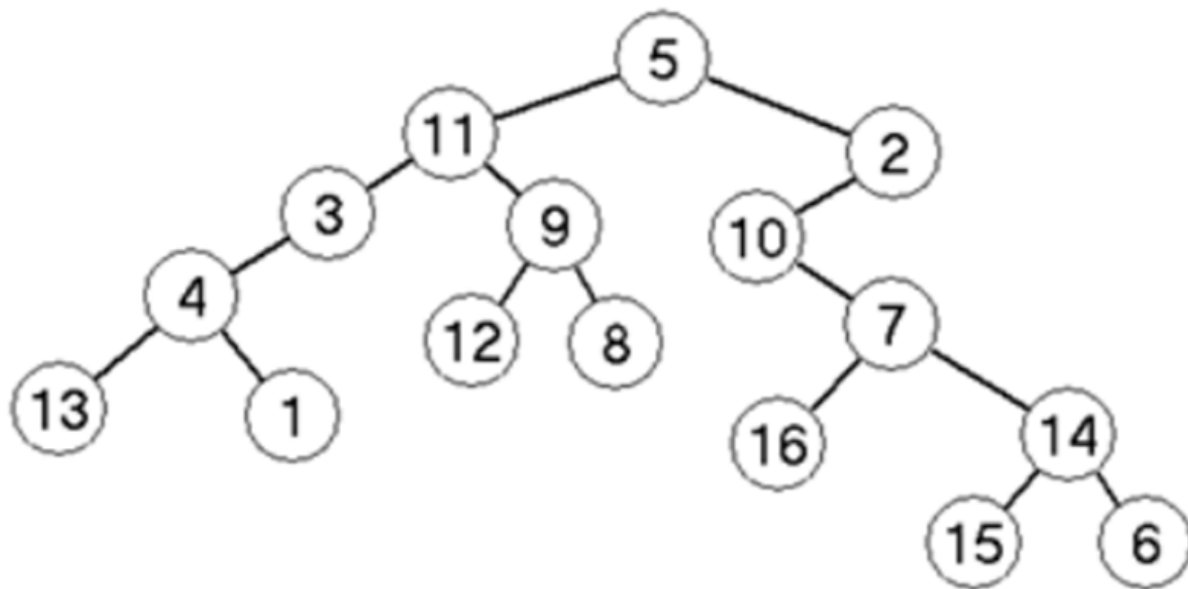


Order obtained by postorder traversal:

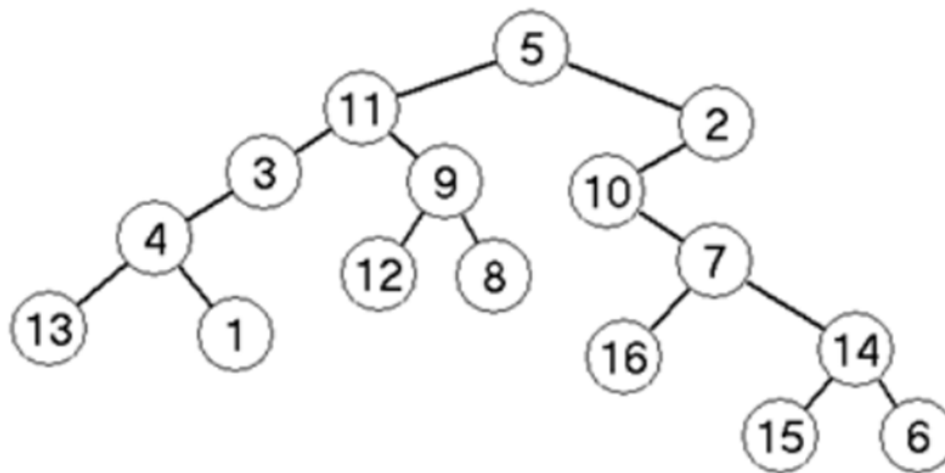
g d k h e b i j f c a

Task: Traversal of Binary Trees 1

- Give the order in which the three recursive traversal algorithms print the keys of the following tree



Task: Traversal of Binary Trees 1



Preorder: 5, 11, 3, 4, 13, 1, 9, 12, 8, 2, 10, 7, 16, 14, 15, 6

Inorder: 13, 4, 1, 3, 11, 12, 9, 8, 5, 10, 16, 7, 15, 14, 6, 2

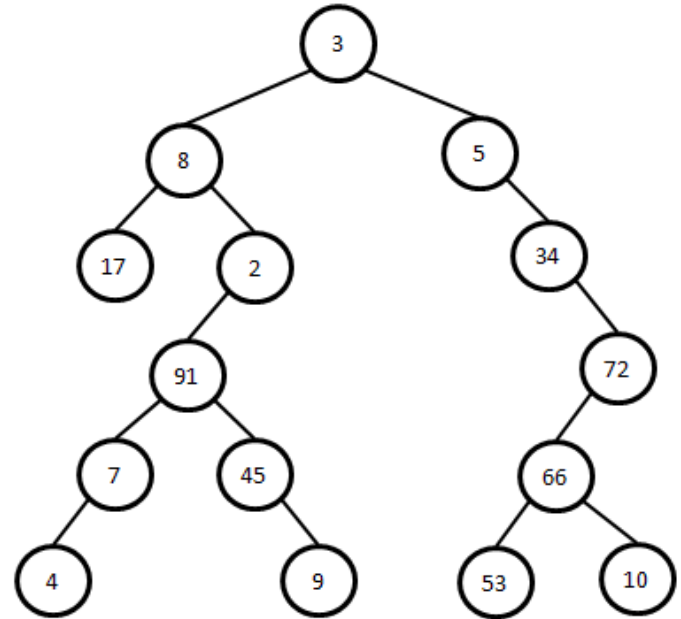
Postorder: 13, 1, 4, 3, 12, 8, 9, 11, 16, 15, 6, 14, 7, 10, 2, 5

Task: Traversal of Binary Trees 2

- Reconstruct the tree from the following postorder and inorder traversals!
 - POSTORDER: 17, 4, 7, 9, 45, 91, 2, 8, 53, 10, 66, 72, 34, 5, 3
 - INORDER: 17, 8, 4, 7, 91, 45, 9, 2, 3, 5, 34, 53, 66, 10, 72

Task: Traversal of Binary Trees 2

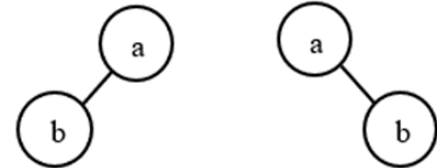
- POSTORDER: 17, 4, 7, 9, 45, 91, 2, 8, 53, 10, 66, 72, 34, 5, 3
- INORDER: 17, 8, 4, 7, 91, 45, 9, 2, 3, 5, 34, 53, 66, 10, 72
- How can this be solved?
 - Let us determine what the root of the tree is.
 - Which traversal tells us this for sure?
 - **Postorder** (prints it last): 3
- What can we determine from the **inorder** traversal?
 - the elements printed to the left of 3 will be in the left subtree of 3,
 - the elements to the right will be in the right subtree.
- Detailed presentation of the solution:



<https://people.inf.elte.hu/veanna/algo1/feladatgyujtemeny/binfa/abr05/megoldas.html>

Can we reconstruct the tree from any two traversals?

- **No!** From which two is it not possible?
- **Answer:** the pair preorder and postorder is not sufficient!
- **Why?**
 - Because in the case of nodes with one child, it cannot be decided whether the child is the left or the right child of the node!

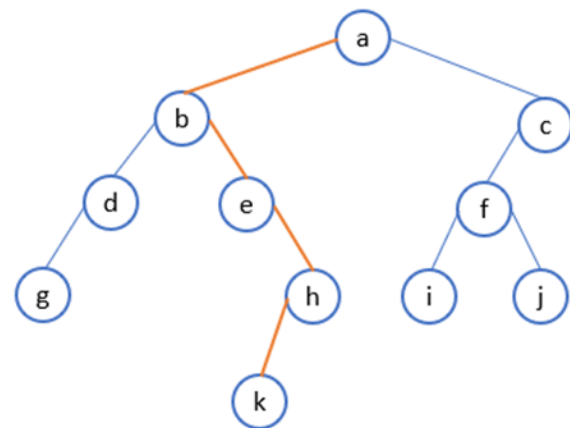


PRE: a b
POST: b a

a b
b a

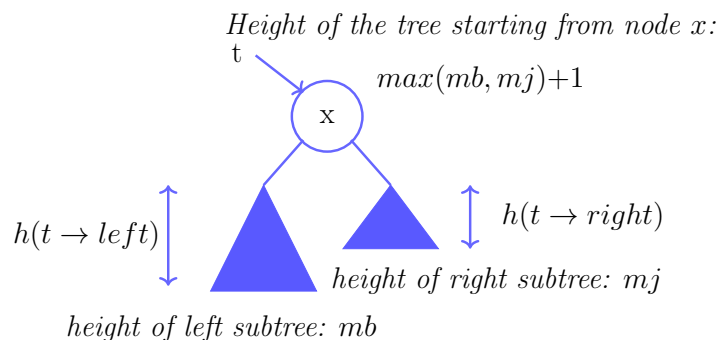
Height of a Binary Tree

- What is meant by **height**:
The length of one of the longest paths of the tree, that is, the number of edges in that path.
- The height of an empty tree is defined as -1 .
- Height of a tree consisting of one node: 0
- Height of a tree consisting of two nodes: 1
- Height of the tree shown in the example: 4

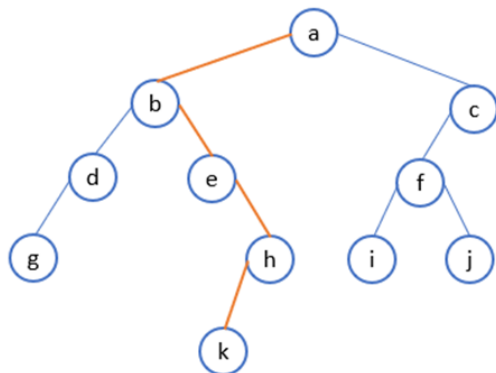


Height of a Binary Tree

- How can we solve it?
- Idea: the height of an arbitrary node x can be determined recursively as follows:
- Determine the height of the left subtree of x , let this be mb .
- Determine the height of the right subtree, let this be mj .
- From this, the height of x is: $\max(mb, mj) + 1$
- Height of an empty tree: -1
- The solution is ready!



Height of Binary Trees



$h(t:\text{Bintree}):Z$

$t \neq 0$	
$mb := h(t \rightarrow \text{left})$	return -1
$mj := h(t \rightarrow \text{right})$	
return $\max(mb, mj) + 1$	

Esetek ellenőrzése:

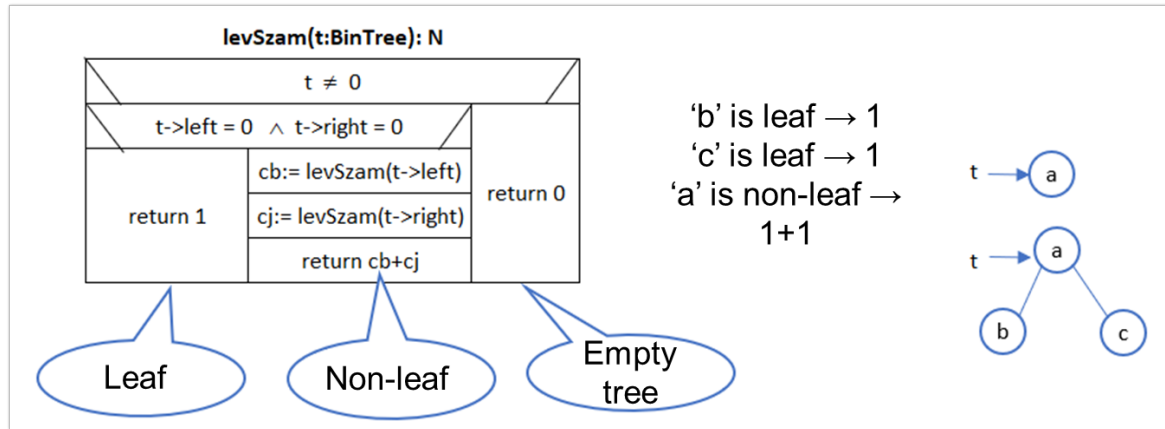
üres fa ($t=0$)

egy pontból álló fa: $t \rightarrow a$

két pontból álló fa: $t \rightarrow a \rightarrow b$

Binary Tree (Tasks)

"Number of Leaves" with a Recursive Algorithm



- Check whether x is a leaf:
 - if yes, then return 1
 - if not, then return the sum of the values obtained from its left and right subtrees

Maximum Key First solution

- **First solution:** Search for a global max using recursive traversal.
- Using any recursive traversal, e.g. following a *preorder* traversal, we “carry along” a max variable, which we continuously update according to the key of the current element. IMPORTANT: max must be a parameter passed by reference!
- Since it cannot provide a solution for an empty tree, the empty-tree case must be filtered out, and we signal an error in that case.

maxKulcs(t:BinTree): T

t ≠ 0	
max:=t->key	Error
preorder_Max(t,max)	
return max	

preorder_Max(t:BinTree, &max:T)

t ≠ 0		
t -> key > max		
max := t->key	skip	skip
preorder_Max(t->left, max)		
preorder_Max(t->right, max)		

- From outside, **maxKulcs** procedure can be called, while **preorder_Max** is only a helper procedure. **maxKulcs** starts the traversal with the key stored in the root of the tree.
- With this method, we can implement any programming theorem! We carry the result variables of the theorem along in the form of reference parameters. The other two traversal algorithms would also be suitable!

Maximum Key Second solution

- **Second solution:** I use the partial solutions of the subtrees.
- We produce the maximum not as a parameter, but as a return value.

maxKulcs(t:BinTree):T	
t=0	
Error	return(maxBejár(t))

maxBejár(t:BinTree):T				
t->left =0 és t->right=0				levél esete
return t->key	t->left≠0 és t->right≠0	t->left≠0 és t->right=0	t->left=0 és t->right≠0	belső pontok esetei
	max1:=maxBejár(t->left)	max1:=maxBejár(t->left)	max1:=maxBejár(t->right)	
	max2:=maxBejár(t->right)			
	max1:=max(max1,max2)			
	return max(max1,t->key)			

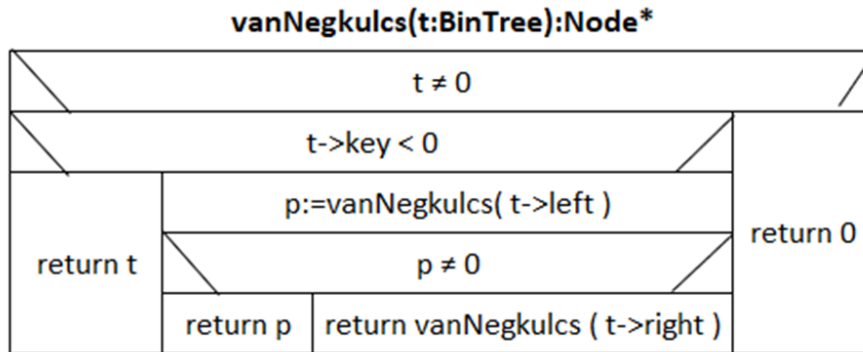
levél esete

belső pontok esetei

- In this case as well, an “initializing” procedure is needed to handle the empty-tree case.
- The precondition of this algorithm is that t is a non-empty tree. Therefore, the recursive call is carried out only on non-empty subtrees. That is why the three possible cases must be handled separately for internal nodes.

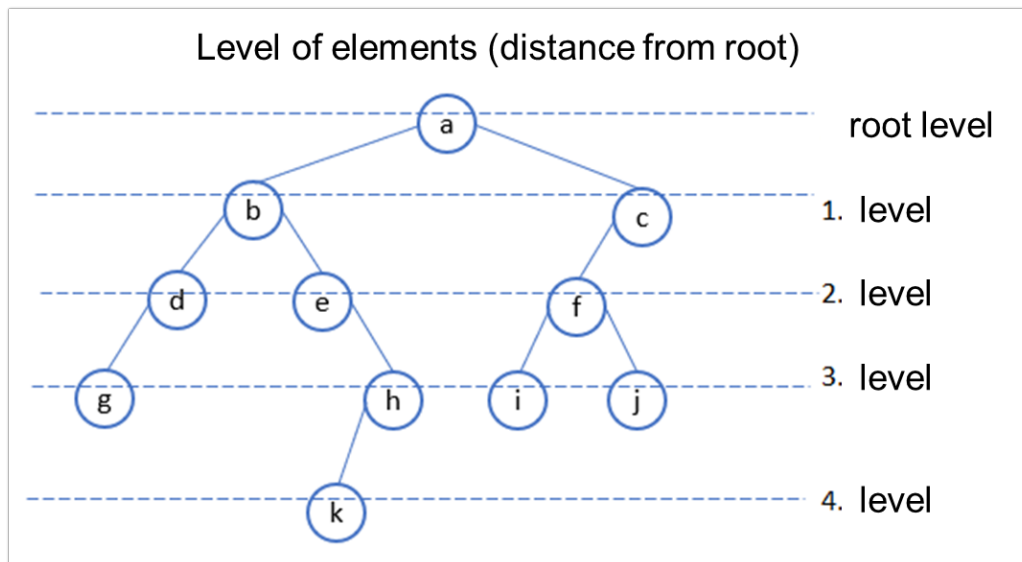
Negative Key Recursively

- **Our task is search.** We implement it as a function. During traversal, if we encounter an element with a negative key, we stop the traversal and return the address of the element. If we traverse the whole tree and do not encounter any element with a negative key, we return zero as the result. Thus there is no need for a logical result variable for the search.



Level-Order Traversal

Levels of a Binary Tree, Tasks Related to Levels



Level-Order Traversal of a Linked Binary Tree

- The algorithm can be found in the notes
- This traversal can also be used as an enumeration, the processing reaches every node of the tree
- The order in which the elements are accessed:

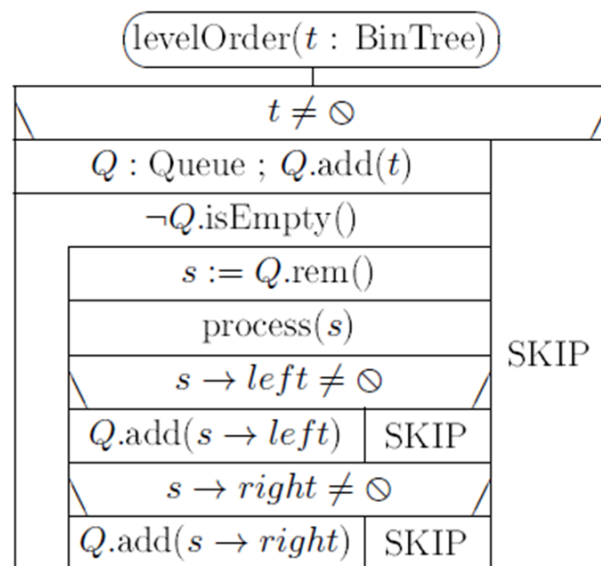
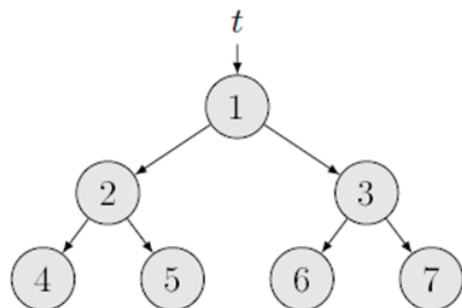
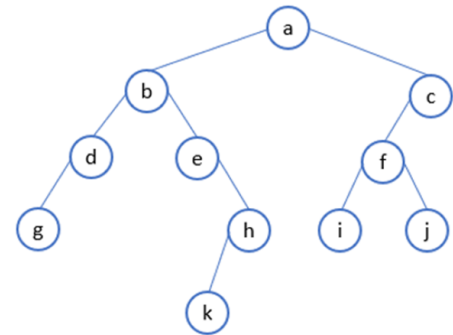


Illustration of Level-Order Traversal

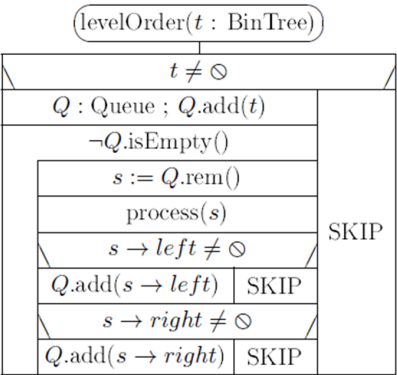


Loop at start

Node

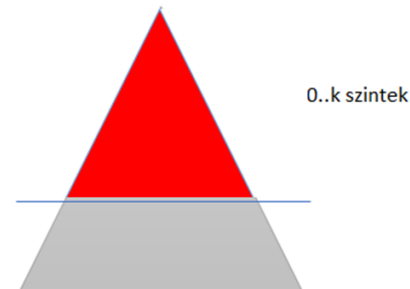
a
b
c
d
e
f
g
h
i,j,k

Q tartalma	
a	
b	c
c	d e
d	e f
e	f g
f	g h i j
g	h i j
h	i j k



How many one-child nodes are there on levels 0..k?

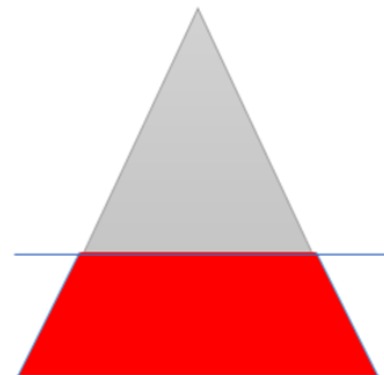
egyGyerek(t: Bintree, k: N) : N		
t = 0 vagy k < 0		
return 0	c1 := egyGyerek(t->left, k-1)	
	c2 := egyGyerek(t->right, k-1)	
	(t->left ≠ 0 és t->right = 0) vagy (t->left = 0 és t->right ≠ 0)	
	return c1+c2+1	return c1+c2



How many one-child nodes are there on levels greater than k ?

egyGyerek(t:Bintree, k: N) : N

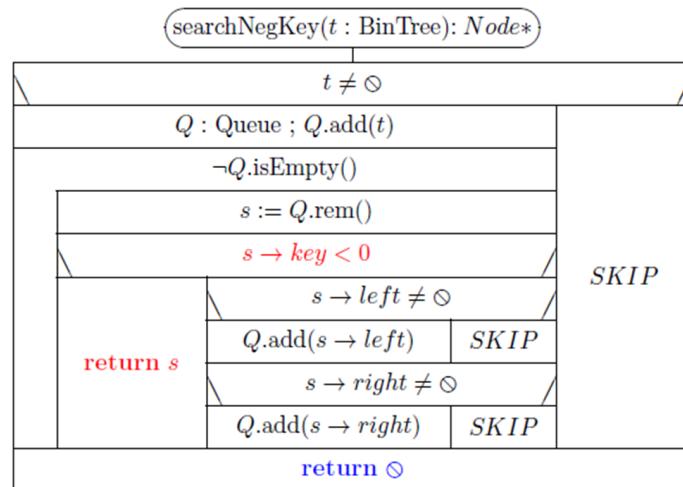
t = 0		
return 0	c1:= egyGyerek(t->left,k-1)	
	c2:=egyGyerek(t->right,k-1)	
	k < 0 és ((t->left≠0 és t->right=0) vagy (t->left=0 és t->right≠0))	
	return c1+c2+1	return c1+c2



k-nál nagyobb szintek

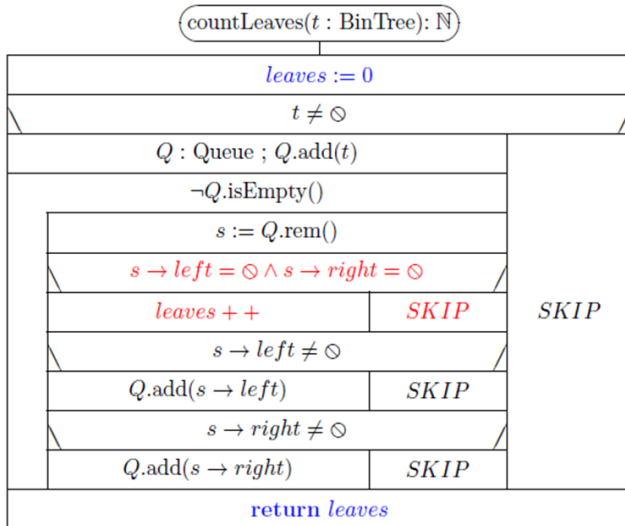
Negative Key by Level-Order Traversal

- The keys of the tree are integers, let us search for an element with a negative key.
- The result of a successful search is a pointer to the element.
- In case of unsuccessful search, we return a NULL pointer.

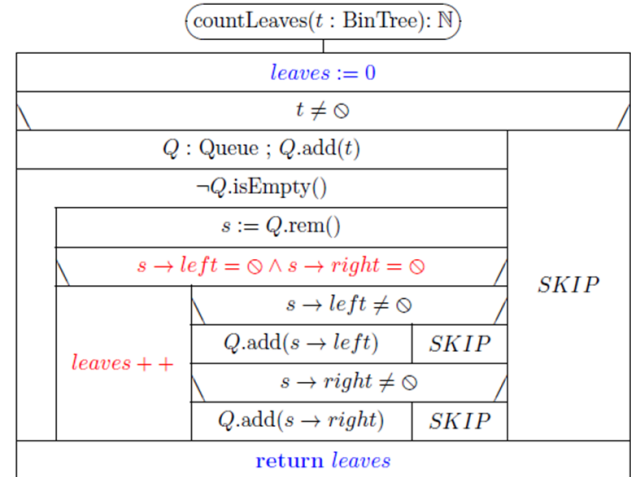


"Number of Leaves" by Level-Order Traversal

- Let us count the leaves of the tree:



process(t) = if t is a leaf, increment the counter



we can do it a bit more cleverly: if t is a leaf, it will not have any descendants anyway

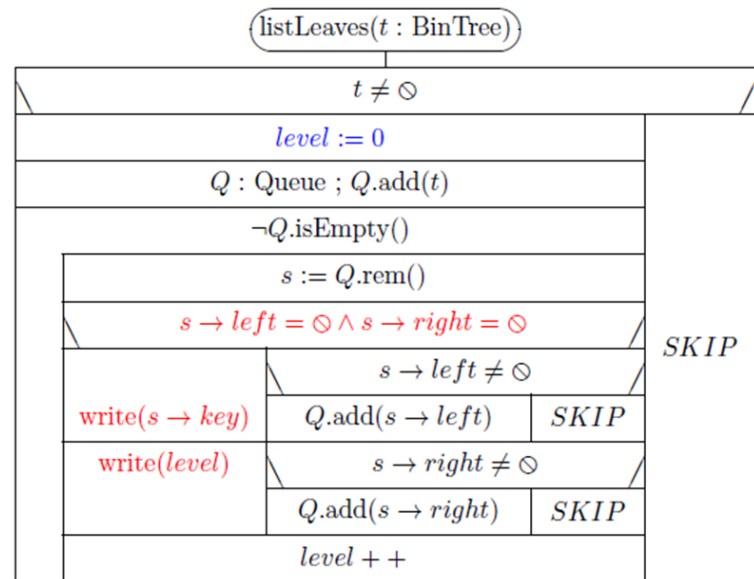
What is the smallest level, where the tree has a leaf?

- **Introductory task:** how can we keep track of the level number of nodes during traversal? Let us first solve the task of listing the keys contained in the leaves of the tree together with the level number of the node.

- The following is an obvious, but incorrect, solution:

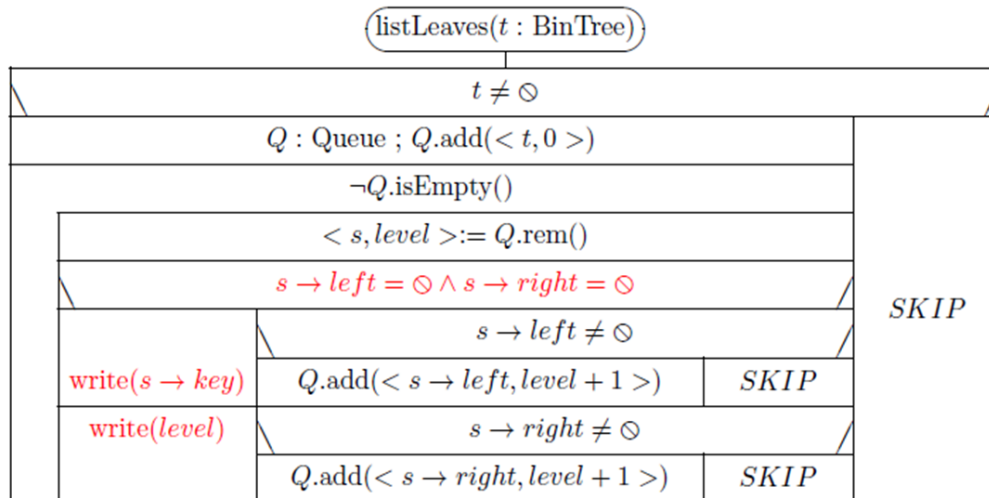
What is the error?

- Instead of the level, it would print the position number of the processed node according to the level-order traversal.



Correction

- We put $\langle \text{Node}^*, N \rangle$ pairs into the queue, storing the level number next to each node as well.

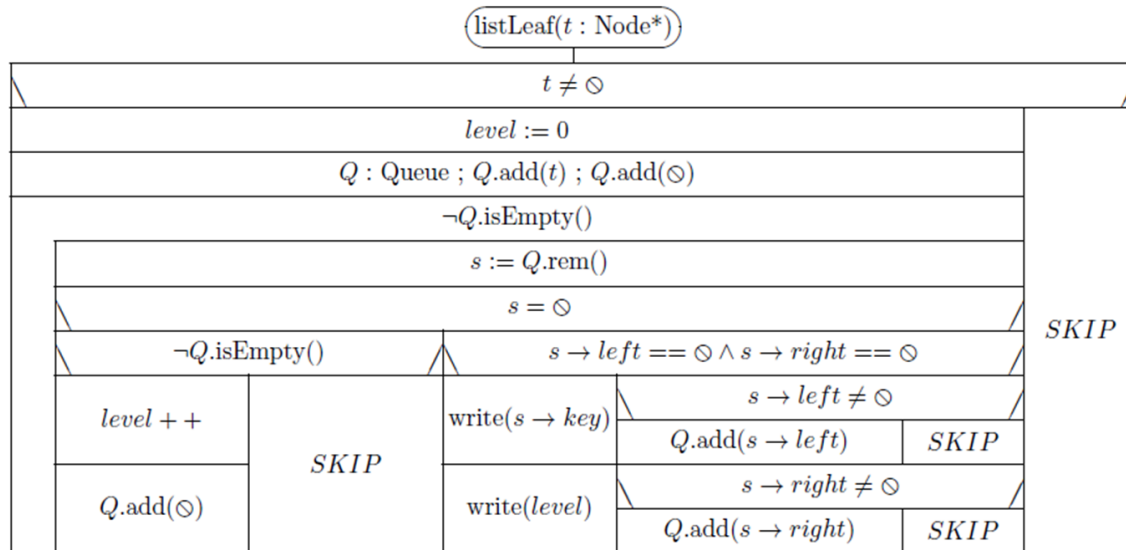


- **Question:** can it also be solved without storing the level number for every node?

Yes, it can be solved, and there are two possible methods:

- First:
 - Using end markers: at the end of each level, we put an end marker (NULL) into the queue. We solve this by initially putting an end marker after the root element, then whenever we take out an end marker, we increase the level and put a new end marker into the queue.
- Second:
 - By counting the number of elements on levels: instead of end markers, we count the elements. We always keep track of the following:
 - a) how many elements are on this level (we calculated this before),
 - b) which element within the level we are currently at,
 - c) we count the number of elements on the next level (the number of elements we put into the queue on this level).

First Method (using End Markers) for Listing Leaves

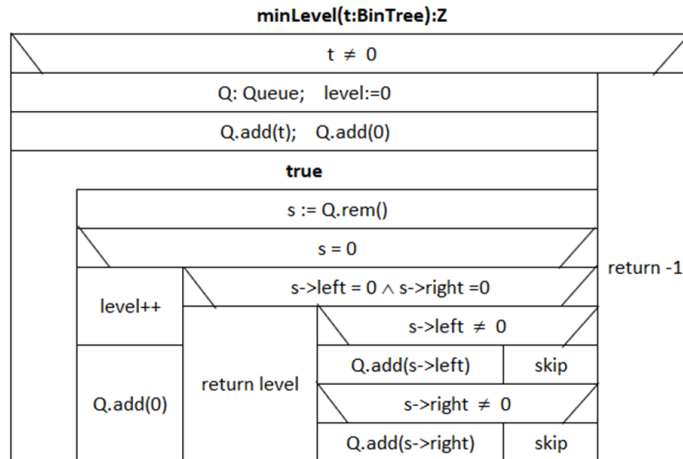


Solving the Original Task with the End-Marker Method

“What is the smallest level where there is a leaf?”

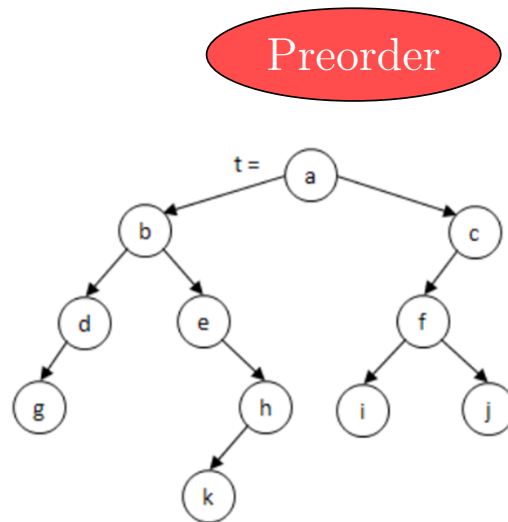
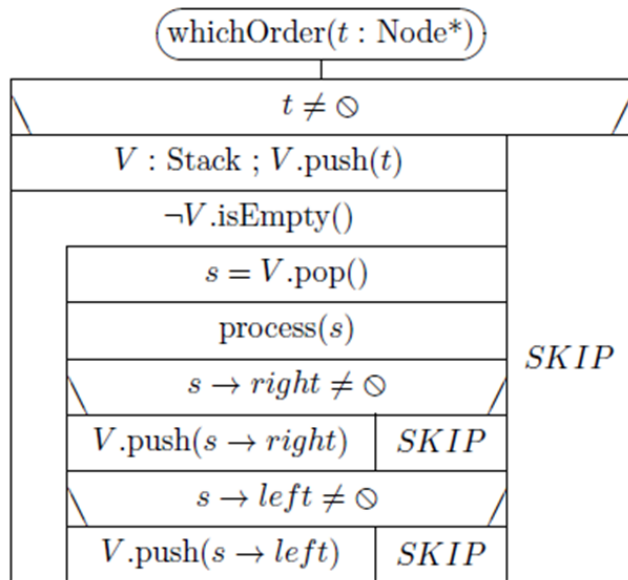
Why is it not an infinite loop?

The task is actually a selection, because if the tree is not empty, it certainly has a leaf, so the level-order traversal is guaranteed to reach a leaf, and since we proceed in level order, this leaf will be one of those closest to the root.



Preorder with stacks

- Let us modify the level-order traversal so that instead of a queue, we use a stack, and we swap the order of processing the children: we first put the right child, then the left child onto the stack (if they exist). Then we obtain the iterative version of one of the recursive traversal algorithms.
- Which one?



Optional Homework

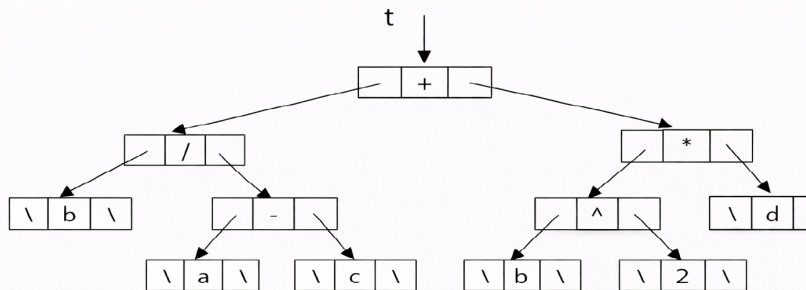
- Given a linked binary tree, write a recursive algorithm (not using level-order traversal) that returns the smallest level number where the tree has a leaf. It must return -1 for an empty tree.
- Given a linked expression tree, write a recursive algorithm which traverses the expression tree and prints the fully parenthesized form of the expression.

Expression Tree

Expression Tree

- Assuming that an arithmetic expression may contain binary, or possibly unary, operation symbols (operators), the expression can be represented by a binary tree; this is called an expression tree.

An Expression tree representing the expression $b/(a-c)+b^2-d$



The postorder traversal of the expression tree results in the expression's Polish notation form:

$b \ a \ c \ - \ / \ b \ 2 \ ^ \ d \ * \ +$

- The expression tree is related to the Polish notation learned earlier, because it can be observed that the Polish notation can be obtained by a **postorder** traversal of the expression tree, and the reverse is also true: the expression tree can be built from the Polish notation.

Building an Expression Tree from the Fully Parenthesized Form

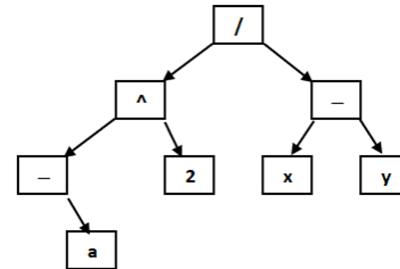
- **Fully (and correctly) parenthesized form:** a form of the original correctly parenthesized expression in which each pair of parentheses contains exactly one operator, along with its corresponding left and right operands. The nested parenthesis pairs determine the order of operations.
- Examples:

- $a+b$ $(a+b)$

- $a+b*c$ $(a+(b*c))$

- $(a+b)/c$ $((a+b)/c)$

- $-a^2/(x-y)$ $(((-a)^2)/(x-y))$



- From the **fully parenthesized** expression, we can easily construct the **expression tree**. We need to find the operator inside the outermost pair of parentheses; this will be the root of the expression tree. Its left subtree is then constructed from the left operand of the operator, and its right subtree from the right operand, in the same way. When an operand is no longer a parenthesized expression, it becomes a leaf.

Building an Expression Tree from Polish Notation

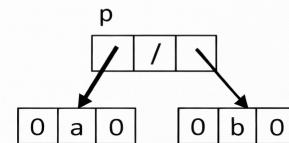
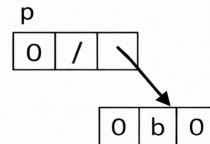
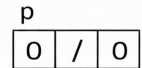
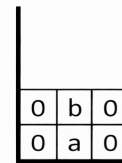
- Let X be a sequence in Polish notation, whose elements (x_i) belong either to the set of operators or to the set of operands. Construct the expression tree from the Polish notation.

treeFromPostfix(X): Node*

V: Stack	
i = 1 to X.length()	
p := new Node (x _i)	
$x_i \in \{\text{operators}\}$	$x_i \in \{\text{operands}\}$
p->right:=V.pop()	skip
p->left:=V.pop()	
V.push(p)	
return V.pop()	

- The stack will contain pointers to the nodes at the top.
- The constructor places the key in the new Node, and sets its left and right pointers to null.
- For an operator, the stack already contains the Node address for the right operand on top and the left operand just below it.
- We push the address of the new Node onto the stack.
- Finally, the stack contains a pointer to the root of the tree.

processing a b /



In reality, the stack contains not Nodes but pointers to Nodes

Thank you for your attention!